WelfareOne

팀 공유 자료 — 무엇이 달라졌고, 데이터는 어떻게 나눌까요

2026년 10월 19일 기준 · 브랜치 `feature/jimin-llm-provider`

1. 한눈에 보기
2. 갱신 데이터 공유 방법 (가이드)
3. main과 비교해 추가된 것과 기대효과
4. 두 브랜치에서 무엇을 적용했나
5. AI를 고르는 규칙 (Gemini / Qwen)
6. 아직 정해야 할 것과 한계
7. 쉬운 용어 풀이

이 문서는 개발 지식이 없어도 읽을 수 있게 썼습니다. 코드 이름은 필요한 곳에만 작게 적었습니다.

1. 한눈에 보기

WelfareOne은 청년이 "월세가 부담돼요"처럼 일상 말투로 물으면 알맞은 복지 정책을 찾아주고, 정책 원문에 근거해 답해주는 서비스입니다. 이번 정리는 **세 가지**입니다.

주제	쉬운 요약
① 검색·화면·답변이 좋아짐	main(기본 버전)보다 관련 없는 정책이 덜 나오고, 후보를 3개씩 넘겨 보고, 정책을 고른 뒤 버튼 한 번으로 질문할 수 있습니다. 서버를 켠 직후 첫 검색이 38초 걸리던 문제도 3.5초로 줄었습니다.
② 두 사람의 작업을 합침	지민님 브랜치(`jimin-search-ux`)를 바탕으로 두고, 종희님 브랜치(`jonghee-search-detail`)에서는 **리랭커(검색 결과 순서를 다시 매기는 도구)**만 가져왔습니다. 겹치는 기능은 중복을 피하려고 가져오지 않았습니다.
③ 데이터 갱신 도구가 생김	복지로에서 **새로 생기거나 바뀐 정책만** 골라 내려받는 프로그램을 만들었습니다. 아직 인증키가 없어 실제로 돌려보지는 않았습니다.

> **숫자로 보면** — main 대비 커밋 28개, 파일 73개 변경. 자동 테스트는 백엔드 185개·프론트 39개가 모두 통과합니다(백엔드 5개는 환경 때문에 건너뛰는 항목).

2. 갱신 데이터 공유 방법 (가이드)

> **먼저 알아둘 점** — 이 가이드의 방식은 **제안**입니다. 아직 팀 합의 전이고, 데이터 수집 도구는 만들어 두었지만 **실제 수집은 실행하지 않았습니다**(복지로 인증키 필요).

2-1. 지금 데이터는 어디에 있나

위치	내용	공유 방법
`data/raw`	복지로에서 받은 정책 원본 파일(XML) 약 1,437개. 마지막 수집은 9월 18~21일	크기가 작아 **Git(코드 저장소)으로 공유**
`data/processed`	원본을 읽기 쉽게 바꾼 파일	Git
`data/chroma`	검색용 데이터베이스(정책 1,418개, 조각 11,850개). 용량이 커서 Git에 올리지 않음	**압축 파일로 따로 공유**(지금처럼)

검색 결과는 `data/chroma`에서 나옵니다. 즉 **원본(raw)만 새로 받아서는 검색에 나타나지 않고**, 검색용 데이터베이스에 다시 넣는 단계가 따로 필요합니다. 이 단계는 프로젝트 규칙상 **팀 합의 후에만** 합니다.

2-2. 갱신분을 따로 분류해서 추가하는 방식 (제안)

핵심 생각은 "**전체를 다시 올리지 말고, 이번에 바뀐 것만 날짜 이름을 붙여 추가한다**"입니다.

1. **점검** — 목록만 받아서 새로 생긴 정책·수정된 정책이 몇 건인지 확인합니다. 아직 아무 파일도 바꾸지 않습니다.

   ```
   python scripts/update_welfare_incremental.py local
   ```

2. **반영** — 바뀐 정책의 상세 파일만 받아 `data/raw`에 넣습니다. 바뀌기 전 옛 파일은 `detail_prev/날짜/`에 보관합니다.

   ```
   python scripts/update_welfare_incremental.py local --apply
   ```

3. **보고서 확인** — `data/raw/local/update_reports/update_날짜.json`에 "추가된 ID, 수정된 ID, 실패, 목록에서 사라진 정책"이 정리됩니다.

4. **Git 브랜치로 공유** — `data/update-날짜` 같은 이름의 브랜치에 올립니다. Git은 바뀐 파일만 보여 주므로 리뷰가 쉽고, 옛 버전도 이력에 남습니다.

5. **팀 합의 후 검색 DB 갱신** — 한 사람(담당자)이 다시 적재하고, 새 `data/chroma`를 `chroma_날짜.zip`과 설명 파일(레코드 수, 임베딩 모델, 기준이 된 업데이트 보고서 날짜)과 함께 공유합니다. 팀원은 폴더를 통째로 교체합니다.

2-3. 폴더 정리 제안

```
data/raw/local/detail/              최신 정책 원본          → Git에 올림
data/raw/local/update_reports/      이번에 무엇이 바뀌었나 → Git에 올림
data/raw/local/detail_prev/날짜/    바뀌기 전 옛 파일       → 내 PC에만 보관 (Git 제외 권장)
data/chroma/                        검색 DB                → Git 제외, 압축 파일로 따로 공유
```

`detail_prev`를 Git에서 빼자는 이유는 Git 이력이 이미 옛 버전을 기억하기 때문입니다(중복 방지). 이 변경은 `.gitignore` 수정이 필요해서 아직 하지 않았습니다.

2-4. 이 도구가 지키는 약속

- 검색 DB(`data/chroma`)는 **읽지도 쓰지도 않습니다.**
- 기본 실행은 "점검만"이고, `--apply`를 붙여야 파일이 바뀝니다.
- 목록에서 사라진 정책은 알려만 주고 지우지 않습니다.
- 인증키는 환경변수나 입력창으로만 받고, 화면·파일에 남기지 않습니다.
- 지자체는 "최종 수정일"을 비교하지만, 중앙부처는 수정일이 없어서 모두 받아 내용이 달라졌는지 비교합니다(하루 호출 한도 때문에 2번에 나눠 끝납니다).

3. main과 비교해 추가된 것과 기대효과

main은 처음 합친 기본 버전입니다. 아래는 그 위에 더해진 것들입니다. 표시: 지민 지민님 브랜치에서 온 것 · 종희 종희님 브랜치에서 온 것 · 추가 합치면서 새로 만든 것

3-1. 검색이 더 정확해졌습니다

추가된 것	쉬운 설명	기대효과 / 확인된 수치
지민 거리 하한	질문과 너무 먼 정책은 결과에서 뺍니다.	"오늘 날씨" 같은 정책 밖 질문은 빈 결과. 평가 질문 280개 중 정답 정책의 99.2%는 그대로 남음
지민 일상 말투 전용 설정	구어체 질문은 더 엄격한 기준으로, 지역보다 내용을 먼저 봅니다.	멀리 있는 지역 정책이 가까운 정책보다 위로 오는 문제 감소
지민 구어체 → 정책 용어	"배고파" → "무료 급식 식사 지원"처럼 AI가 검색어를 바꿔 줍니다.	짧은 말도 검색됨
지민 AI 관련도 재판정	찾은 후보를 AI가 한 번 더 "이 사람에게 도움이 되나?" 확인해 아니면 뒤로 보냅니다.	기본 화면 관련 정책 비율 41% → 63% (질문 15개 소규모 측정). 실패해도 오류 없이 기존 순서로 동작
지민 검색 안정화	"자격증관련 정책 알려줘"처럼 띄어쓰기·말투가 달라도 같은 결과가 나오게 합니다.	말투 때문에 빈 결과가 나오던 문제 해결
종희 리랭커 추가 연결	검색 결과 후보를 질문과 얼마나 맞는지 다시 읽고 순서를 매기는 별도 AI 모델(`bge-reranker-v2-m3`)입니다.	모호한 질문에서 중앙부처 후보가 눈에 띄게 좋아짐(예: "자격증 시험 비용" 질문에서 '어업활동지원'이 빠지고 '전문기술인재장학금'이 올라옴). 소규모 확인이라 정량 비교는 앞으로 필요

3-2. 화면이 더 편해졌습니다 지민

추가된 것	쉬운 설명
후보 3개씩 이전/다음	후보가 많으면 지자체·중앙부처 그룹마다 3개씩 쪽을 넘겨 봅니다. (후보는 최대 12개까지)
관련도 낮은 후보 구분	관련도가 낮아 보이는 정책은 지우지 않고 뒤에 "관련도 낮음" 배지를 달아 이어서 보여 줍니다.
자주 묻는 질문 버튼	정책을 고르면 "지원 대상 / 금액 / 신청 방법 / 기간 / 서류 / 문의처" 버튼이 나옵니다. 버튼 질문은 분석 단계를 건너뛰어 더 빠릅니다.
상담 제목 직접 수정	왼쪽 상담 목록에서 ✎ 버튼으로 제목을 고칩니다.
키 입력 안내 추가	서버가 AI를 제공하면 "본인 Gemini 키 입력"이 필수가 아닌 선택으로 바뀝니다.

3-3. 답변이 더 읽기 쉬워졌습니다 지민

- **질문 유형별 답변 구성** — 지원 대상은 "누가 대상 → 제외·우선 대상 → 확인할 점"처럼 소제목과 불릿으로 정리하고, 원문에 없는 내용은 지어내지 않고 "확인되지 않습니다"로 안내합니다.
- **자료 링크 추출 확대** — `www…`나 도메인만 적힌 주소도 바로가기 버튼으로 만들고, "링크 알려줘"에 모순된 답이 나오지 않게 보정했습니다.

3-4. 더 빠르고 튼튼해졌습니다 추가

변화	설명과 수치

서버 시작 때 미리 준비(워밍업)	켠 직후 첫 검색이 **38초 → 3.5초**. 첫 사용자가 오래 기다리다 화면이 끊기던 문제를 없앴습니다.
GPU(그래픽카드) 사용	리랭커 단계가 CPU에서는 질문당 5.5~7.4초였지만 GPU에서는 **0.6~1.0초**. GPU가 없는 PC는 자동으로 느리지만 안전한 방식으로 동작합니다.
공용 모듈로 정리 지민	.env 읽기, AI 재시도, 검색 흐름을 한곳에 모아 같은 규칙으로 동작합니다.
AI 분석 결과 보정 추가	AI가 정책 이름도 없이 "특정 정책"이라 분류하거나, "알려줘·정책" 같은 군더더기를 검색 키워드로 넣어도 코드가 바로잡습니다. 어떤 AI를 써도 같은 규칙이 적용됩니다.
리랭커/AI가 실패해도 계속 동작	모델이 없거나 응답이 늦으면 오류 대신 기존 검색 순서로 결과를 보여 줍니다.

3-5. AI 종류를 바꿀 수 있는 구조가 생겼습니다 추가

지금까지는 모든 AI 호출이 Gemini에 고정돼 있었습니다. 이제 질문 분석·판정·답변이 "AI 호출 창구" 하나(`LLMClient`)만 거치도록 바꿔서, **Gemini 말고 서버가 직접 돌리는 Qwen 같은 AI도 꽂을 수 있습니다.** 설정하지 않으면 지금과 똑같이 동작합니다. 자세한 규칙은 5장에 있습니다.

3-6. 팀 작업을 돕는 것들 추가

- 데이터 증분 갱신 도구(2장), 검색 거리 측정·사람 라벨링용 시트 생성 스크립트 지민
- 계약서(공통 약속 문서) 수정 **제안서** — 팀 합의 전이라 계약서 본문은 건드리지 않고 제안 문서(`docs/contract_update_proposal.md`)로만 정리했습니다.
- 프로젝트 동작 설명서·라벨링 가이드(PDF), 브랜치 개요 문서

4. 두 브랜치에서 무엇을 적용했나

> 두 사람이 같은 목표(관련 있는 정책만 보여주기)를 서로 다른 방법으로 구현했습니다. 그대로 합치면 **같은 일을 하는 코드가 두 벌**이 돼서 AI를 두 번 부르거나 서로 충돌합니다. 그래서 **지민님 브랜치를 바탕으로 두고, 종희님 쪽에서는 겹치지 않는 부분(리랭커)만 가져왔습니다.**

4-1. 종희님 브랜치 `feature/jonghee-search-detail`

종희님이 만든 것	적용 여부	이유
리랭커 모듈 `reranker_service.py`	적용	지민님 쪽에 없던 기능이라 그대로 가져옴. 검색 후보의 순서를 AI가 다시 매깁니다.
리랭커 설정값, `reranker_score` 필드, `transformers` 의존성	적용	나중에 종희님 브랜치와 합칠 때 충돌하지 않도록 **같은 내용**으로 맞춤
리랭커 테스트	적용	그대로 가져옴
Gemini 후보 적합성 판정 `relevance_service.py`	미적용	지민님의 AI 관련도 재판정(`relevance_judge.py`)과 **목적이 같음**. 하나만 남기기로 하고 지민님 것을 유지
제외 의도 구조화(`exclusions`)	미적용	지민님은 "제외 키워드" 방식으로 이미 점수에 반영 중이라 방식이 충돌
검색 서비스 대폭 변경 (후보 30개, 거리 한도 0.70, 지역 가산점 제거)	미적용	지민님의 일상 말투 전용 설정·키워드 점수와 충돌. 대신 리랭커가 쓸 후보 수만 따로 조절
화면의 관련성 배지(직접/조건부/간접), "검증 못 함" 경고	미적용	서버가 그 값을 만들지 않는 구조라 화면만 가져오면 빈 값이 됨
계약서 본문 직접 수정	미적용	계약서는 팀 합의 없이 바꾸지 않는 문서 → 제안서로 대체

종희님 방향 위에 지민님 화면만 얹은 **별도 브랜치**(`feature/jimin-frontend-on-jonghee`, 아직 내 PC에만 있고 필요하면 올립니다)도 만들어 두었습니다. 팀이 종희님 방식을 기준으로 삼기로 하면 그 브랜치를 쓸 수 있습니다.

4-2. 지민님 브랜치 `feature/jimin-search-ux`

지민님이 만든 것	적용 여부
검색 안정화, 거리 하한, 일상 말투 전용 설정, 키워드 점수	전부 적용 (바탕)
AI 관련도 재판정, 검색 흐름 공용 모듈	전부 적용
3개씩 쪽 넘김, 관련도 낮음 구분, 상담 제목 수정	전부 적용
자주 묻는 질문 버튼, 질문 유형별 답변 구성	전부 적용
라벨링 시트 스크립트, 가이드·개요 문서, 테스트	전부 적용

4-3. 병합 결과 (충돌 확인 완료)

> 종희님 브랜치의 **이력은 병합 기록에 포함**했고, 내용은 위 표대로 리랭커만 채택했습니다(병합 커밋 `de59862`).
> 종희님 브랜치의 나머지 코드는 삭제된 것이 아니라 **종희님 브랜치에 그대로 남아 있습니다.**

병합 순서	충돌
지민 브랜치를 main에 먼저 병합 → 이후 종희 브랜치 병합	없음 (이미 포함돼 새로 들어오는 변경도 없음)

종희 브랜치를 main에 먼저 병합 → 이후 지민 브랜치 병합	없음 (결과가 지민 브랜치 내용과 동일)

어느 쪽이 먼저 main에 들어가도 충돌은 없습니다. 다만 **종희님 방식(재판정 서비스·제외 의도 구조화 등)은 main에 반영되지 않는다**는 점은 팀이 알고 있어야 하므로, 이 부분은 6장의 "정해야 할 것"에 둡니다.

4-4. 합치면서 새로 한 일

1. 종희님 리랭커를 지민님 검색 흐름의 "검색 직후, AI 재판정 직전"에 끼워 넣었습니다. (순서: 검색 → 리랭커 → AI 재판정 → 화면)
2. 리랭커가 처리할 후보 수를 장치에 맞게 자동 조절했습니다 (CPU 12개 / GPU 30개).
3. 서버 워밍업, GPU 설치 안내, AI 호출 창구 통일, Qwen 연결, 키 입력 안내, 데이터 갱신 도구를 추가했습니다.

5. AI를 고르는 규칙 (Gemini / Qwen)

사용자가 자기 Gemini 키를 넣어서 쓰는 기능은 **그대로 둡니다.** 거기에 "서버가 직접 돌리는 Qwen"이 기본 AI가 될 수 있게 했습니다. 요청마다 아래 순서로 정해집니다.

순서	조건	사용하는 AI
1	사용자가 본인 Gemini 키를 보냄	**항상 그 키의 Gemini** (서버 설정과 무관)
2	키 없음 + 서버에 Qwen이 설정돼 있음	서버의 Qwen
3	키 없음 + 서버 Gemini 키 사용을 허용함	서버의 Gemini
4	위에 모두 해당 없음	"API 키가 필요합니다" 안내

- 사용자 키로 보낸 질문이 서버 Qwen으로 새어 나가지 않도록 자동 테스트로 고정했습니다.
- 설정을 하나도 안 하면 지금까지와 똑같이 동작합니다.

Qwen 모델 시험 결과 (이 PC, 소규모 확인)

모델	한 번 호출 시간	평가
qwen3-4b-2507 (작은 모델)	1.6~3.5초	**부적합** — 제외 키워드를 엉뚱하게 채움(예: "월세가 부담돼요"에 '주택', '전세'를 제외). 이 값은 검색 점수를 깎아서 위험
qwen3-8b	1.0~4.2초	**권장** — 검색어·제외 키워드가 정확. 다만 답변에 지침 문구가 섞이거나 판정이 가끔 시간 안에 못 끝남
(참고) Gemini	검색 전체 약 3~6초	기준. Qwen 8B는 검색 전체가 약 5~11초

> **주의** — 질문 5개 수준의 확인이라 "Qwen을 기본으로 올려도 된다"는 결론은 아직 낼 수 없습니다. 평가 질문 280개로 Gemini와 비교한 뒤 정해야 합니다. 또 LM Studio에서 모델이 미리 올라가 있지 않으면 첫 요청이 100초 넘게 걸립니다.

6. 아직 정해야 할 것과 한계

팀이 정해야 할 것

1. **검색 DB 다시 적재 여부** — 데이터를 새로 받아도 검색 DB를 다시 만들기 전에는 검색에 나오지 않습니다. 전체를 다시 만들지, 바뀐 정책만 넣을지, 누가 실행하고 어떻게 공유할지.
2. **종희님/지민님 방식 정리** — 겹치는 기능(재판정 두 벌, 제외 의도 방식)을 장기적으로 어느 쪽으로 통일할지.
3. **계약서 8장 제안 승인** — AI 선택 규칙, `GET /config`, 환경변수.
4. **Qwen을 기본 AI로 올릴 기준** — 평가 질문 280개에서 어느 수준이면 올릴지.
5. **Qwen 실행 위치** — 지금은 이 PC의 LM Studio. 사용자가 늘면 전용 서버나 vLLM 같은 방식으로 옮길 수 있음 (코드는 주소만 바꾸면 됨).

한계 (솔직하게)

- 데이터 갱신 도구는 **시험용 자료로만 검증**했고, 실제 복지로 서버에는 아직 연결해 보지 않았습니다.
- 검색 품질 수치 중 일부는 질문 15개~75개 수준의 작은 표본이며, 관련 여부 판정도 사람이 아닌 AI가 했습니다. "방향을 보는 용도"입니다.

- 리랭커 효과는 사례로는 보이지만, 평가 질문으로 켠/끈 결과를 수치로 비교한 것은 아닙니다(쉬운 평가셋이라 차이가 크게 나지 않았음).
- 특정 집단 전용 정책(장애인·보훈 등)은 질문에 언급이 없으면 접히는데, 이 방식이 맞는지는 자격 판정 기능과 함께 다시 설계해야 합니다.
- 화면 동작은 개발자가 직접 써 보며 확인한 범위이고, 여러 사용자가 동시에 쓰는 시험은 하지 않았습니다.

7. 쉬운 용어 풀이

용어	뜻
브랜치	코드를 안전하게 따로 고치는 "복사본 작업 공간". 다 되면 본 줄기(main)에 합칩니다.
main	팀이 함께 쓰는 기본(본 줄기) 버전.
임베딩	문장의 의미를 숫자로 바꾸는 기술. 숫자가 가까울수록 뜻이 비슷하다고 보고 검색합니다.
ChromaDB (`data/chroma`)	정책 문서를 임베딩과 함께 저장해 둔 검색용 데이터베이스.
거리(distance)	질문과 정책이 얼마나 다른지 나타내는 값. 작을수록 비슷합니다.
리랭커	1차로 찾은 후보를 질문과 하나씩 맞대어 읽고 순서를 다시 매기는 AI 모델. 검색의 "2차 심사".
Gemini / Qwen	글을 이해하고 쓰는 AI. Gemini는 구글의 외부 서비스(API 키 필요), Qwen은 서버에서 직접 돌릴 수 있는 모델.
API 키	외부 서비스를 쓰기 위한 비밀번호 같은 값. 코드나 채팅에 올리면 안 됩니다.
LM Studio	Qwen 같은 모델을 내 PC에서 돌리는 프로그램.
GPU	그래픽카드. AI 계산을 CPU보다 훨씬 빠르게 합니다.
BYOK	"내 키를 가져와서 쓰기". 사용자가 자기 Gemini 키를 넣어 쓰는 방식.
증분 수집	전체를 다시 받지 않고, 새로 생기거나 바뀐 것만 받는 방식.
재적재	받은 원본으로 검색 DB를 다시 만들어 넣는 작업. 시간이 걸리고 모두의 DB가 바뀌므로 합의 후 한 사람이 진행합니다.
워밍업	서버를 켠 직후 미리 한 번 돌려 두어, 첫 사용자가 오래 기다리지 않게 하는 준비 작업.

> **문의** — 이 문서의 근거 문서: `docs/contract_update_proposal.md` (계약서 수정 제안), `docs/BRANCH_OVERVIEW.md` (지민님 브랜치 변경 정리), `scripts/README.md` (스크립트 사용법).